

Step 2 — A Supervisor Routes Between Them — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (the real LangGraph/ `Command` shapes), **Advanced** (what a Supervisor actually needs to route a 2-specialist, multi-step task correctly). Read Basic first even if you already built Project 4 — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — State, nodes, and the routing decision](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — State, nodes, and the routing decision

Basic Version

A LangGraph graph needs 3 things here: a shared state shape everyone reads and writes, a node for each specialist, and a Supervisor node that looks at the state so far and decides who goes next.

Things to use: - A `TypedDict` for the shared state — the same idea as Project 4's shared state, just with fields for notes findings and web findings instead of a brief. - `Command(goto=..., update=...)` — the object a node returns to say "go here next, and update state with this." - `StateGraph(YourStateType)`, with `.add_node(...)` for each node and `.set_entry_point("supervisor")`.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

`TeamState` needs enough fields that the Supervisor can tell, just by looking at state, what's already been done — that's what makes routing a real decision instead of a guess. `notes_findings: str` and `web_findings: str` start as empty strings; once a specialist runs, its node fills in its own field and nothing else.

The exact pieces: - `from langgraph.graph import StateGraph, END` - `from langgraph.types import Command` - Each specialist node signature: `def notes_agent_node(state: TeamState) -> Command:` — calls `run_notes_agent(state["task"])` from Step 1, and returns `Command(goto="supervisor", update={"notes_findings": result})`. Notice it routes back to `"supervisor"`, not straight to `END` — the Supervisor is the one that decides when the team is actually done. - The Supervisor node: `def supervisor_node(state: TeamState) -> Command:` — a small decision function (a plain `if / elif` chain checking which fields are still empty is enough here; you don't need an LLM call just to decide "has notes_findings been filled in yet"). - `graph = StateGraph(TeamState)`, `graph.add_node("supervisor", supervisor_node)`, `graph.add_node("notes_agent", notes_agent_node)`, `graph.add_node("web_agent", web_agent_node)`, `graph.set_entry_point("supervisor")`, `app = graph.compile()`.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Think hard about task 3 from **A Real Example** — the one that needs *both* specialists. The Supervisor can't just be "call one specialist, then finish" — it needs to genuinely decide, based on the task's wording and what's already been filled in, whether more work is needed before it's done. A simple, honest way to do this without adding a whole second LLM call just for routing: have the Supervisor check the task text itself for both "notes-shaped" and "web-shaped" signals (does it mention notes? does it mention a URL or fetching a page?), track which specialists it decided the task actually needs, and only route to `END` once every specialist it decided were needed have filled in their field.

Also think about what happens if `notes_agent_node` or `web_agent_node` gets called a second time by mistake — because your routing logic has an off-by-one bug, say. Since each node re-runs Step 1's full agent (a whole new MCP connection, a whole new model call), a routing bug here isn't just wasteful, it can also silently overwrite a specialist's real result with a redundant duplicate call's result. Guard against this directly: before a specialist node does its real work, it's fine for it to trust the Supervisor's routing (that's the Supervisor's job to get right), but the Supervisor itself should track "have I already sent this to `notes_agent`?" explicitly, in a state field like `visited: list[str]`, not just infer it from whether `notes_findings` is empty (an agent could legitimately return an empty string if it found nothing — that's not the same as "hasn't been called yet").

Things to try before Hint 2: - Trace task 1 (notes only) through your routing logic by hand, on paper, before running it — confirm it visits `supervisor → notes_agent → supervisor → END` and never touches `web_agent` at all. - Trace task 3 (both) the same way — confirm it visits both specialists exactly once each, in some order, before `END`.

Difference between Basic, Intermediate, and Advanced: Basic names the 3 pieces. Intermediate gives the real `Command / StateGraph` shapes for the simple case. Advanced is about getting the routing decision actually correct for the multi-specialist case — telling "needs both" from "needs one," and telling "hasn't run yet" from "ran and legitimately found nothing" — which is the difference between a Supervisor that happens to work on your first test and one that's actually making a real decision.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
TeamState: task, notes_findings, web_findings, needed (list of which specialists

supervisor_node(state):
    figure out which specialists the task needs, the first time through
    if a needed specialist hasn't run yet: route to it
    else: route to END

notes_agent_node(state):
    call run_notes_agent(state["task"])
    save result into notes_findings, mark notes_agent as visited
    route back to supervisor

web_agent_node(state): same idea, for the web agent

graph: supervisor is the entry point, wired to both specialist nodes and back
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
# state.py
from typing import TypedDict

class TeamState(TypedDict):
    task: str
    needed: list[str]
    visited: list[str]
    notes_findings: str
    web_findings: str
```

```
# supervisor.py
from langgraph.types import Command
from state import TeamState

def _figure_out_needed(task: str) -> list[str]:
    needed = []
    task_lower = task.lower()
    if "note" in task_lower:
        needed.append("notes_agent")
    if "http" in task_lower or "fetch" in task_lower or "page" in task_lower:
        needed.append("web_agent")
    if not needed:
        needed.append("notes_agent") # default: assume notes if nothing else matches
    return needed

def supervisor_node(state: TeamState) -> Command:
    needed = state.get("needed") or _figure_out_needed(state["task"])
    visited = state.get("visited") or []

    for specialist in needed:
        if specialist not in visited:
            return Command(goto=specialist, update={"needed": needed, "visited":
            visited})

    return Command(goto="__end__")
```

Your turn: write `notes_agent_node` and `web_agent_node` in `notes_agent.py` / `web_agent.py`, each appending its own name to `visited` and writing its own findings field, then wire `graph.py` together and compare against the [Solution](#).

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Fill in the specialist node and graph-wiring, using this skeleton:

```
# notes_agent.py (node function, added to Step 1's file)
from langgraph.types import Command
from state import TeamState
# run_notes_agent(task) -> str already exists from Step 1

def notes_agent_node(state: TeamState) -> Command:
    result = run_notes_agent(state["task"])
    visited = state.get("visited", []) + ["notes_agent"]
    return Command(
        goto="supervisor",
        update={"notes_findings": result, "visited": visited},
    )
```

```
# graph.py
from langgraph.graph import StateGraph
from state import TeamState
from supervisor import supervisor_node
from notes_agent import notes_agent_node
from web_agent import web_agent_node

def build_graph():
    graph = StateGraph(TeamState)
    graph.add_node("supervisor", supervisor_node)
    graph.add_node("notes_agent", notes_agent_node)
    graph.add_node("web_agent", web_agent_node)
    graph.set_entry_point("supervisor")
    return graph.compile()

if __name__ == "__main__":
    app = build_graph()
    # your turn: run app.invoke({"task": "...", "needed": [], "visited": [], "not
    # on tasks 1, 2, and 3, and print the final state each time
```

Run task 3 and print the final state's `visited` field — confirm it contains both `"notes_agent"` and `"web_agent"`, in whatever order the Supervisor picked, then compare against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic sketches the state fields and routing idea in plain words. Intermediate gives a real, working `supervisor_node` with a simple keyword-based "what does this task need" check. Advanced wires the whole graph together and shows exactly how a specialist's node hands control back to the Supervisor with `Command(goto="supervisor", ...)`, instead of routing straight to `END` itself — keeping the Supervisor, not each specialist, in charge of deciding when the team's work is actually finished.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)